

MSc FUNDAMENTALS OF PROGRAMMING

Capstone Project Exercise

Item	Details
Programming language	Python
Dataset workbook	MSc_Fundamentals_Programming_Complete_Capstone_Datasets.xlsx
Project allocation	Each group completes one assigned project
Assessed concepts	Algorithms, variables, decisions, loops, data structures, functions, validation, testing, documentation and Git
Not required	Pandas, NumPy, charts, machine learning or analysis-result exports

Purpose

The capstone requires students to translate a realistic data problem into a structured, modular and tested Python program. The programming challenge should come from algorithm design, validation and use of core Python rather than advanced libraries.

Available projects

No.	Project	Dataset worksheet
1	Health Facility Data-Quality Auditor	01_Health_Quality
2	Student Performance and Intervention System	02_Student_Performance
3	Agricultural Market-Price Analyser	03_Market_Prices
4	Medicine Inventory Monitoring System	04_Medicine_Inventory
5	Weather Observation Analysis Tool	05_Weather_Readings
6	Retail Sales Performance Analyser	06_Retail_Sales
7	Household Water-Consumption and Billing System	07_Water_Consumption
8	Road-Accident Records Analyser	08_Road_Accidents
9	Personal Expense and Budget Tracker	09_Expense_Tracker
10	Dataset Metadata and Quality Assessment Tool	10_Dataset_Metadata

Common menu-control requirement

Each project must provide an appropriate menu. Implement the menu using a while loop, input(), if-elif-else and break. Each option must call a separate function. An invalid selection must display an error and return to the menu.

```
while True:
    display_menu()
    choice = input("Enter selection: ").strip()
    if choice == "1":
        perform_first_operation()
    elif choice == "2":
        perform_second_operation()
    elif choice == "7":
        print("Program closed.")
```

```
    break  
else:  
    print("Invalid selection.")
```

Common dataset-quality rule

The datasets contain deliberate anomalies, including missing values, negative values, out-of-range values, unsupported categories or inconsistent capitalization. Students must standardize a value only when the question defines a justified mapping. Values that cannot be safely corrected must be rejected and reported with a reason.

Project 1: Health Facility Data-Quality Auditor

Item	Details
Dataset worksheet	01_Health_Quality
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A county health department receives facility reports containing expected records, complete records, timeliness and error counts. Some records are incomplete, negative or logically inconsistent.

Question

Using the dataset in the 01_Health_Quality worksheet, develop a Python program that performs the tasks below.

Validation rules

- Facility ID and name are provided.
- Expected records are greater than zero.
- Complete records and error count are not negative.
- Complete records do not exceed expected records.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Validate every record and display rejected records with reasons.
- Calculate completeness = $\text{complete_records} / \text{expected_records} \times 100$.

- Classify completeness as Excellent ($\geq 90\%$), Satisfactory ($75\% - < 90\%$) or Needs improvement ($< 75\%$).
- Count facilities in each classification.
- Identify the highest and lowest completeness.
- Count on-time submissions.
- Display total, valid and invalid record counts and a formatted summary.

Required menu

Option	Action
1	View facility records
2	Analyse facility completeness
3	View facilities requiring improvement
4	View invalid records
5	View dataset summary
6	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 2: Student Performance and Intervention System

Item	Details
Dataset worksheet	02_Student_Performance
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A university needs to calculate weighted performance and identify students who require academic intervention.

Question

Using the dataset in the 02_Student_Performance worksheet, develop a Python program that performs the tasks below.

Validation rules

- Marks and attendance are numeric values from 0 to 100.
- Student ID and programme are provided.
- Project-submission status is a Boolean value.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Validate every student record and report invalid values.
- Calculate final mark = 20% assignment + 20% CAT + 60% examination.
- Assign A (≥ 70), B ($60 < 70$), C ($50 < 60$), D ($40 < 50$) or E (< 40).
- Identify intervention cases: final mark below 50, attendance below 75%, or project not submitted.
- Calculate class average.

- Identify highest and lowest final marks.
- Count students in every grade.
- Display a performance and intervention summary.

Required menu

Option	Action
1	View valid students
2	View invalid records
3	Analyse class performance
4	View intervention list
5	Search for a student
6	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 3: Agricultural Market-Price Analyser

Item	Details
Dataset worksheet	03_Market_Prices
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

Farmers and traders need to compare prices across five correctly located markets. Every one of four commodities has a baseline record in every market. Prices may be expressed per kilogram or per 90-kilogram bag.

Question

Using the dataset in the 03_Market_Prices worksheet, develop a Python program that performs the tasks below.

Validation rules

- Record ID, market and commodity are provided.
- Unit resolves to kg or 90kg bag.
- Price is numeric and greater than zero.
- Quantity is numeric and not negative.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Standardize recognized alternatives: wakulima → Wakulima; Nakuru top market → Nakuru Top Market; KG/Kgs/kilogram → kg; 90 kg bag → 90kg bag. Do not guess unsupported values.
- Reject invalid records and display reasons.

- Calculate price per kg; divide a 90kg-bag price by 90.
- Convert bag quantities to kilograms by multiplying by 90.
- Display unique commodities alphabetically.
- For a selected commodity, calculate minimum, maximum and average price per kg.
- Identify cheapest and most expensive markets and count prices above/below average.
- Sort market records from lowest to highest price.
- Compare a selected commodity in two markets and report the price difference and cheaper market.
- Display record-quality and quantity summaries.

Required menu

Option	Action
1	View available commodities
2	View prices for a commodity
3	Analyse a commodity
4	Compare two markets
5	View invalid records
6	View dataset summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 4: Medicine Inventory Monitoring System

Item	Details
Dataset worksheet	04_Medicine_Inventory
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A health facility needs to calculate closing stock, identify shortages and flag expired medicine.

Question

Using the dataset in the 04_Medicine_Inventory worksheet, develop a Python program that performs the tasks below.

Validation rules

- Medicine ID and name are provided.
- Opening stock, receipts, issues and reorder level are non-negative.
- Expired status is Boolean.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Validate all records and report invalid values.
- Calculate closing stock = opening stock + quantity received – quantity issued.
- Flag records producing negative closing stock.
- Classify as Expired, Out of stock, Low stock or Adequate.
- Display medicines requiring reorder.
- Identify medicine with highest closing stock.

- Calculate total closing stock.
- Count medicines in every status category.

Required menu

Option	Action
1	View inventory
2	Process stock status
3	View reorder list
4	View expired medicines
5	View invalid records
6	View inventory summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 5: Weather Observation Analysis Tool

Item	Details
Dataset worksheet	05_Weather_Readings
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

Weather stations record temperature, rainfall, humidity and wind speed. Some readings are missing or outside reasonable ranges.

Question

Using the dataset in the 05_Weather_Readings worksheet, develop a Python program that performs the tasks below.

Validation rules

- Temperature is between -10°C and 50°C .
- Rainfall and wind speed are zero or greater.
- Humidity is between 0% and 100%.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Validate readings and explain every rejection.
- Calculate average temperature, rainfall, humidity and wind speed.
- Identify highest and lowest temperature.
- Identify the station with highest rainfall.
- Count temperatures above 30°C .
- Classify temperatures as Cold ($<20^{\circ}\text{C}$), Moderate ($20\text{--}30^{\circ}\text{C}$) or Hot ($>30^{\circ}\text{C}$).

- Count readings in each classification.
- Search for a selected station and display its records.

Required menu

Option	Action
1	View valid readings
2	Analyse weather
3	Search by station
4	View temperature classifications
5	View invalid records
6	View summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 6: Retail Sales Performance Analyser

Item	Details
Dataset worksheet	06_Retail_Sales
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A retailer requires transaction, branch, product and salesperson performance summaries.

Question

Using the dataset in the 06_Retail_Sales worksheet, develop a Python program that performs the tasks below.

Validation rules

- Sale ID, branch and product are provided.
- Quantity and unit price are numeric and greater than zero.
- Payment method is recognized.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Standardize recognized capitalization differences in branch names.
- Validate transactions and report rejected records.
- Calculate transaction value = quantity $\times$ unit price.
- Calculate total and average transaction value.
- Identify highest-value transaction.
- Identify branch, product and salesperson generating the highest revenue.
- Count transactions by payment method.

- Display transactions exceeding a user-provided threshold.

Required menu

Option	Action
1	View transactions
2	Analyse sales
3	Compare branches
4	View salesperson performance
5	View invalid records
6	View summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 7: Household Water-Consumption and Billing System

Item	Details
Dataset worksheet	07_Water_Consumption
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A water utility needs to calculate customer consumption and bills from previous and current meter readings.

Question

Using the dataset in the 07_Water_Consumption worksheet, develop a Python program that performs the tasks below.

Validation rules

- Meter readings are present and non-negative.
- Current reading is not below previous reading.
- Billing band is Band A, Band B or Band C.
- Fixed charge is not negative.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Validate records and display rejection reasons.
- Calculate consumption = current reading – previous reading.
- Apply Band A = KES 40, Band B = KES 55 and Band C = KES 70 per unit.
- Calculate bill = consumption × rate + fixed charge.
- Classify consumption as Low (<50), Moderate (50–100) or High (>100).

- Identify highest-consumption customer.
- Calculate total consumption and total billing.
- Compare domestic and commercial consumption.

Required menu

Option	Action
1	View customer records
2	Calculate bills
3	View high-consumption customers
4	Compare customer types
5	View invalid records
6	View billing summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 8: Road-Accident Records Analyser

Item	Details
Dataset worksheet	08_Road_Accidents
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A road-safety agency needs to identify accident patterns, risky locations and leading causes.

Question

Using the dataset in the 08_Road_Accidents worksheet, develop a Python program that performs the tasks below.

Validation rules

- Vehicles involved and casualties are non-negative integers.
- Severity is Minor, Serious or Fatal.
- Location, cause and time period are provided.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Standardize recognized capitalization differences in accident causes.
- Validate records and display reasons for rejection.
- Calculate total accidents and casualties.
- Count accidents by severity and cause.
- Identify most common cause.
- Identify location with most accidents and location with most casualties.
- Identify time period with most accidents.

- Calculate the percentage of fatal accidents.

Required menu

Option	Action
1	View accident records
2	Analyse severity
3	Analyse causes
4	Analyse locations and time periods
5	View invalid records
6	View safety summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 9: Personal Expense and Budget Tracker

Item	Details
Dataset worksheet	09_Expense_Tracker
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

An individual needs a menu-driven program to monitor income, expenses, budgets and payment methods.

Question

Using the dataset in the 09_Expense_Tracker worksheet, develop a Python program that performs the tasks below.

Validation rules

- Transaction type is Income or Expense.
- Amount is numeric and greater than zero.
- Expense category is provided.
- Budget limit is not negative.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Standardize recognized transaction-type alternatives only when a mapping is defined.
- Validate transactions and report rejected records.
- Calculate total income, expenditure and balance.
- Calculate expense totals by category.
- Compare category expenditure with budget limits.

- Warn when a category exceeds its budget.
- Identify the highest-spending category.
- Count transactions by payment method.

Required menu

Option	Action
1	View valid transactions
2	Add or search a transaction
3	View income and expenditure summary
4	View budget warnings
5	View invalid records
6	View payment summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 10: Dataset Metadata and Quality Assessment Tool

Item	Details
Dataset worksheet	10_Dataset_Metadata
Core output	Menu-driven Python program
Libraries	Core Python only

Scenario

A data-science team maintains metadata about datasets obtained from surveys, sensors and administrative systems.

Question

Using the dataset in the 10_Dataset_Metadata worksheet, develop a Python program that performs the tasks below.

Validation rules

- Records and variables are greater than zero.
- Missing and duplicate counts are non-negative.
- Missing and duplicate counts do not exceed total records.
- Source type is recognized.

Required tasks

- Prepare pseudocode or a flowchart before coding.

Coding Tasks

- Store the records using an appropriate data structure.
- Validate metadata and explain every rejection.
- Calculate missing-data percentage = $\text{missing values} / \text{number of records} \times 100$.
- Classify quality as Excellent ($<5\%$), Satisfactory ($5\% - <10\%$) or Needs improvement ($\geq 10\%$).
- Identify dataset with most records, most variables and highest missing-data percentage.
- Calculate total records represented by valid datasets.

- Count datasets by source type.
- Count and warn about datasets containing personal data.
- Sort datasets from highest to lowest quality.

Required menu

Option	Action
1	View dataset metadata
2	Assess data quality
3	Search for a dataset
4	View personal-data warnings
5	View invalid records
6	View portfolio summary
7	Exit

The menu must repeat until Exit is selected. Invalid selections must display an error message and return to the menu. Each non-exit option must call a suitable function.

Project 11: Employee Attendance & Overtime Analyzer

Scenario

An organization records employee attendance, hours worked, hourly pay rates, employment categories and approved leave days. Management requires a Python program to identify attendance problems, calculate overtime payments and summarise employee performance. The supplied synthetic records include missing, inconsistent and logically invalid values that the program must handle safely.

Required tasks

- i. Prepare pseudocode or a flowchart before coding. It must show inputs, validation, decisions, repetition, calculations and outputs.
- ii. Represent the supplied records using suitable core-Python data structures
- iii. Develop a function that validates every employee record before it is analysed. Accept a record only when the employee ID and name are present; the department is present; expected days are greater than zero; present and leave days are non-negative; present days plus leave days do not exceed expected days; hours worked are non-negative; hourly rate is greater than zero; and employment type is recognised. Display every invalid record together with its rejection reason. Exclude invalid records from all calculations.
- iv. Attendance analysis: For every valid employee, calculate absence days and adjusted attendance percentage. Approved leave must not count as unexplained absence. Classify attendance as Excellent (at least 95%), Satisfactory (85% to below 95%), or Needs improvement (below 85%).
- v. Overtime and payment calculations: Use standard monthly hours of 160 for Full-time, 80 for Part-time and 120 for Contract employees. Calculate overtime hours, overtime payment at 1.5 times the hourly rate, regular payment and total payment. Overtime hours cannot be negative.
- vi. Employee performance findings: Identify the employees with the highest and lowest attendance, the most overtime hours, the highest overtime payment and the highest total payment. Clearly handle ties if they occur.
- vii. Organizational summary: Calculate total and average overtime hours, total overtime expenditure, average attendance, the number of employees in each attendance category, the number with unexplained absences, and counts of supplied, valid and invalid records.
- viii. Department comparison: Calculate average attendance and overtime expenditure for each department. Identify the department with the highest overtime expenditure and present department results in a clear text-based format.

Required menu

Implement the following menu. Each selection must call an appropriate function.

EMPLOYEE ATTENDANCE AND OVERTIME ANALYSER

1. View all valid employee records
2. Search for an employee
3. Analyse employee attendance
4. View employees requiring attendance improvement
5. Analyse overtime
6. Compare departments
7. View invalid records
8. View organisational summary
9. Exit

Testing requirements

Document at least five completed tests in tests.md. At minimum, cover normal, invalid, boundary, search and menu-control behaviour.

Test	Input or condition	Expected result	Actual result	Status
Normal	Valid record	Processed successfully	Student completes	Pass/Fail
Missing/invalid	Required value missing or invalid	Rejected with reason	Student completes	Pass/Fail
Boundary	Value exactly on a classification limit	Correct category	Student completes	Pass/Fail
Search	Existing and non-existing item	Correct records/message	Student completes	Pass/Fail
Menu	Invalid option and Exit	Menu repeats/terminates correctly	Student completes	Pass/Fail

Git requirements

- Initialize the Git repository at the beginning.
- Make at least five meaningful commits showing progressive development.
- Use descriptive commit messages.
- Include the Python source, README, tests and algorithm.
- Inspect history using `git log --oneline`.
- Tag the final version as v1.0.
- Do not commit passwords, keys, personal data or unnecessary system files.

```
git init
git add README.md
git commit -m "Initialise capstone project"
```

```
git add main.py
git commit -m "Implement validation functions"

git log --oneline
git tag -a v1.0 -m "Final capstone project submission"
```

Recommended repository structure

```
project-name/
├── README.md
├── main.py
├── tests.md
├── docs/
│   └── algorithm.pdf
└── .gitignore
```

Deliverables

- Python source file or files.
- README.md with problem, objectives, functions, validation rules and running instructions.
- Pseudocode or flowchart.
- tests.md with at least five completed tests.
- Git repository link or complete repository folder.
- Evidence of commit history and the v1.0 tag.
- Short demonstration and individual code defence.

Assessment guide

Component	Marks
Problem definition and objectives	5
Algorithm: pseudocode or flowchart	10
Data structures and representation	10
Validation and handling invalid records	15

Component	Marks
Functions and modularity	15
Conditions, loops and menu control	15
Correct calculations and summaries	10
Testing and reliability	10
Git, documentation and code quality	10
Total	100